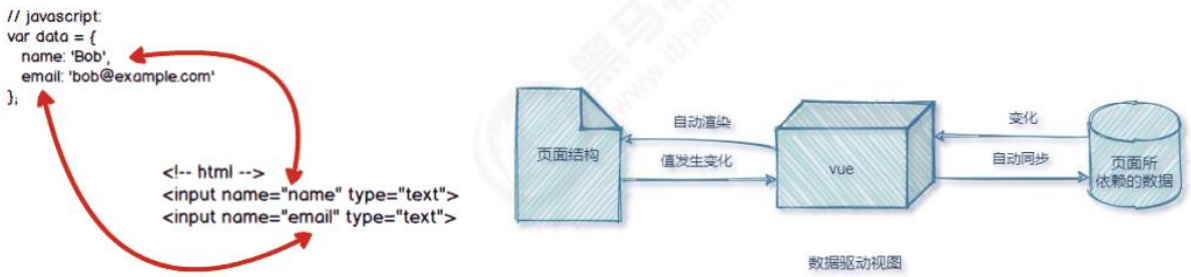
- **vue简介**
  - 什么是vue
    - 官方给出的概念：Vue (读音 /vjuː/，类似于 view) 是一套用于构建用户界面的前端框架。

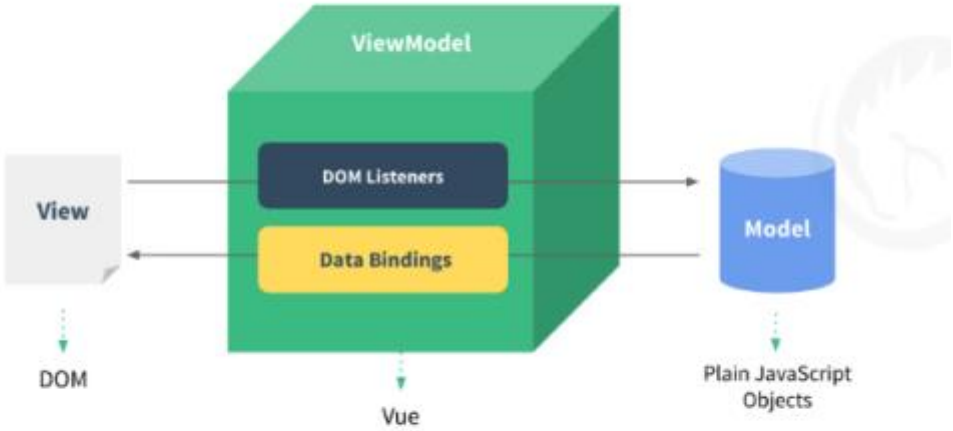
  - vue的特性
    - vue框架的特性，主要体现在如下两方面：
      - 数据驱动视图
      - 双向数据绑定
    - 数据驱动视图
      - 在使用了vue的页面中，vue会监听数据的变化，从而自动重新渲染页面的结构。示意图如下：

        - 好处：当页面数据发生变化时，页面会自动重新渲染！
        - 注意：数据驱动视图是单向的数据绑定。
    - 双向数据绑定
      - 在填写表单时，双向数据绑定可以辅助开发者在不操作 DOM 的前提下，自动把用户填写的内容同步到数据源中。示意图如下：



        - 好处：开发者不再需要手动操作 DOM 元素，来获取表单元素最新的值
    - MVVM

- MVVM 是 vue 实现数据驱动视图和双向数据绑定的核心原理。MVVM 指的是 Model、View 和 ViewModel，它把每个 HTML 页面都拆分成了这三个部分，如图所示：



MVVM 示意图

- 在 MVVM 概念中：
  - Model 表示当前页面渲染时所依赖的数据源。
  - View 表示当前页面所渲染的 DOM 结构。
  - ViewModel 表示 vue 的实例，它是 MVVM 的核心。

- MVVM 的工作原理
  - ViewModel 作为 MVVM 的核心，是它把当前页面的数据源（Model）和页面的结构（View）连接在了一起。

MVVM 的工作原理

  - 当数据源发生变化时，会被 ViewModel 监听到，VM 会根据最新的数据源自动更新页面的结构
  - 当表单元素的值发生变化时，也会被 VM 监听到，VM 会把变化过后最新的值自动同步到 Model 数据源中

- vue 的版本
  - 当前，vue 共有 3 个大版本，其中：
    - 2.x 版本的 vue 是目前企业级项目开发中的主流版本
    - 3.x 版本的 vue 于 2020-09-19 发布，生态还不完善，尚未在企业级项目开发中普及和推广
    - 1.x 版本的 vue 几乎被淘汰，不再建议学习与使用
  - 总结：
    - 3.x 版本的 vue 是未来企业级项目开发的趋势；
    - 2.x 版本的 vue 在未来（1 ~ 2年内）会被逐渐淘汰；

- **vue的基本使用**
  - 基本使用步骤
    - 导入 vue.js 的 script 脚本文件
    - 在页面中声明一个将要被 vue 所控制的 DOM 区域
    - 创建 vm 实例对象（vue 实例对象）

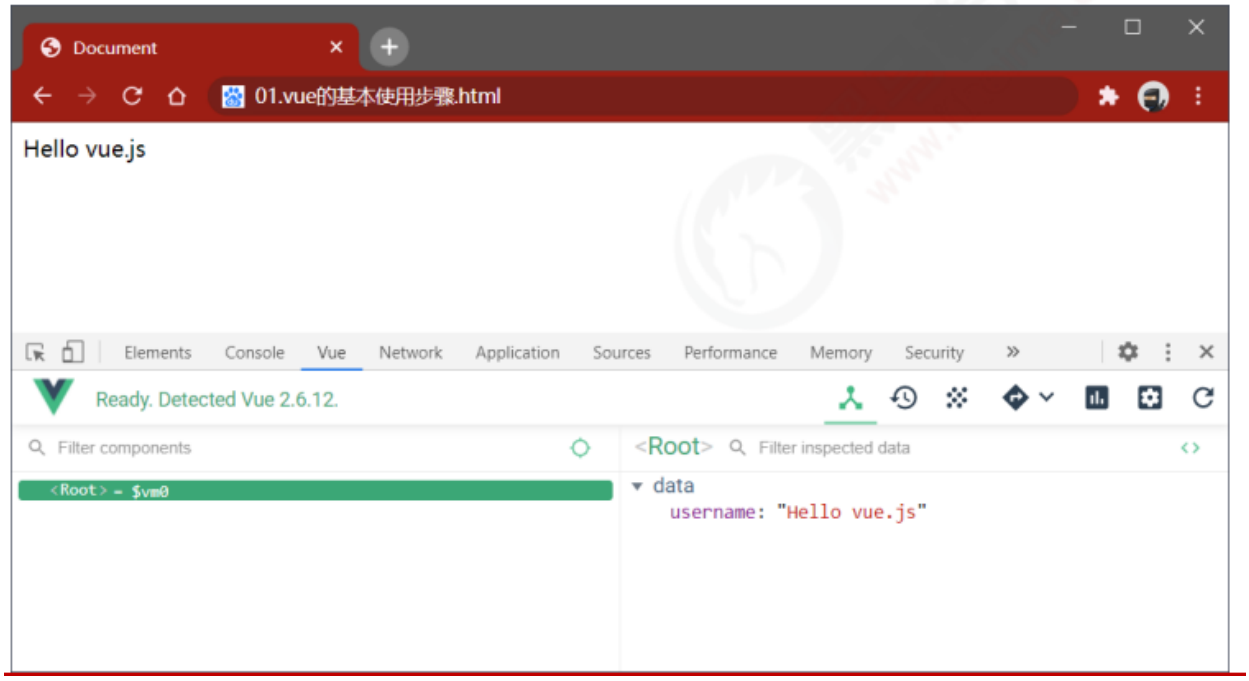
```
<body>
  <!-- 2. 在页面中声明一个将要被 vue 所控制的 DOM 区域 -->
  <div id="app">{{username}}</div>

  <!-- 1. 导入 vue.js 的 script 脚本文件 -->
  <script src="./lib/vue-2.6.12.js"></script>
  <script>
    // 3. 创建 vm 实例对象（vue 实例对象）
    const vm = new Vue({
      // 3.1 指定当前 vm 实例要控制页面的哪个区域
      el: '#app',
      // 3.2 指定 Model 数据源
      data: {
        username: 'zs'
      }
    })
  </script>
</body>
```

  - 基本代码与 MVVM 的对应关系

- **vue的调试工具**
  - 安装 vue-devtools 调试工具
    - vue 官方提供的 vue-devtools 调试工具，能够方便开发者对 vue 项目进行调试与开发。
    - Chrome 浏览器在线安装 vue-devtools： https://chrome.google.com/webstore/detail/vuejs-devtools/nhdogjmejiglipccpnnnanhbledajbpd
    - FireFox 浏览器在线安装 vue-devtools： https://addons.mozilla.org/zh-CN/firefox/addon/vue-js-devtools/
  - 配置 Chrome 浏览器中的 vue-devtools
    - 点击 Chrome 浏览器右上角的"…"按钮，选择更多工具 -> 扩展程序 -> Vue.js devtools 详细信息，并勾选如下 的两个选项：

注意：修改完配置项，须重启浏览器才能生效！

  - 使用 vue-devtools 调试 vue 页面
    - 在浏览器中访问一个使用了 vue 的页面，打开浏览器的开发者工具，切换到 Vue 面板，即可使用 vue-devtools调试当前的页面。



- **vue的指令与过滤器——指令的概念**
  - 指令（Directives）是 vue 为开发者提供的模板语法，用于辅助开发者渲染页面的基本结构。
  - vue 中的指令按照不同的用途可以分为如下 6 大类：

        - 内容渲染指令
        - 属性绑定指令
        - 事件绑定指令
        - 双向绑定指令
        - 条件渲染指令
        - 列表渲染指令
    - 注意：指令是 vue 开发中最基础、最常用、最简单的知识点。

- **vue的指令与过滤器——内容渲染指令**
    - 内容渲染指令用来辅助开发者渲染 DOM 元素的文本内容。常用的内容渲染指令有如下 3 个：
        - v-text
        - {{ }}
        - v-html
    - v-text
        - 用法示例：

```
<!-- 把 username 对应的值，渲染到第一个 p 标签中 -->
<p v-text="username"></p>

<!--- 把 gender 对应的值，渲染到第二个 p 标签中 -->
<!-- 注意：第二个 p 标签中，默认的文本 “性别” 会被 gender 的值覆盖掉 -->
<p v-text="gender">性别</p>
```

        - 注意：v-text 指令会覆盖元素内默认的值。
    - {{ }} 语法
        - vue 提供的 {{ }} 语法，专门用来解决 v-text 会覆盖默认文本内容的问题。这种 {{ }} 语法的专业名称是插值表达式（英文名为：Mustache）。

```
<!-- 使用 {{ }} 插值表达式，将对应的值渲染到元素的内容节点中， -->
<!-- 同时保留元素自身的默认值 -->
<p>姓名：{{username}}</p>
<p>性别：{{gender}}</p>
```

        - 注意：相对于 v-text 指令来说，插值表达式在开发中更常用一些！因为它不会覆盖元素中默认的文本内容。
    - v-html
        - v-text 指令和插值表达式只能渲染纯文本内容。如果要把包含 HTML 标签的字符串渲染为页面的 HTML 元素，则需要用到 v-html 这个指令：

```
<!-- 假设 data 中定义了名为 discription 的数据，数据的值为包含 HTML 标签的字符串： -->
<!-- '<h5 style="color: red;">我在黑马程序员学习 vue.js 课程。</h5>' -->

<p v-html="discription"></p>
```

- 最终渲染的结果为：

姓名：zs

性别：男

我在黑马程序员学习 vue.js 课程。

- **vue的指令与过滤器——属性绑定指令**
  - 如果需要为元素的属性动态绑定属性值，则需要用到 v-bind 属性绑定指令。用法示例如下：

```
<!-- 假设有如下的 data 数据：
data: {
    inputValue: '请输入内容',
    imgSrc: 'https://cn.vuejs.org/images/logo.png'
}
-->

<!-- 使用 v-bind 指令，为 input 的 placeholder 动态绑定属性值 -->
<input type="text" v-bind:placeholder="inputValue" />
<br />
<!-- 使用 v-bind 指令，为 img 的 src 动态绑定属性值 -->
<img v-bind:src="imgSrc" alt="" />
```

  - 属性绑定指令的简写形式
    - 由于 v-bind 指令在开发中使用频率非常高，因此，vue 官方为其提供了简写形式（简写为英文的 : ）。

```
<!-- 假设有如下的 data 数据：
data: {
    inputValue: '请输入内容',
    imgSrc: 'https://cn.vuejs.org/images/logo.png'
}
-->

<!-- 使用 v-bind 指令，为 input 的 placeholder 动态绑定属性值 -->
<input type="text" :placeholder="inputValue" />
<br />
<!-- 使用 v-bind 指令，为 img 的 src 动态绑定属性值 -->
<img :src="imgSrc" alt="" />
```

- 使用 Javascript 表达式
  - 在 vue 提供的模板渲染语法中，除了支持绑定简单的数据值之外，还支持 Javascript 表达式的运算（复杂的语法不支持），例如：

```
{{ number + 1 }}

{{ ok ? 'YES' : 'NO' }}

{{ message.split('').reverse().join('') }}

<div v-bind:id="'list-' + id"></div>
```

- **vue的指令与过滤器——事件绑定指令**
  - vue 提供了 v-on 事件绑定指令，用来辅助程序员为 DOM 元素绑定事件监听。语法格式如下：

```
<h3>count 的值为：{{count}}</h3>
<!-- 语法格式为 v-on:事件名称="事件处理函数的名称" -->
<button v-on:click="addCount">+1</button>
```

  - 注意：原生 DOM 对象有 onclick、oninput、onkeyup 等原生事件，替换为 vue 的事件绑定形式后，分别为：v-on:click、v-on:input、v-on:keyup
  - 通过 v-on 绑定的事件处理函数，需要在 methods 节点中进行声明：

```
const vm = new Vue({
    el: '#app',
    data: { count: 0 },
    methods: {       // v-on 绑定的事件处理函数，需要声明在 methods 节点中
        addCount() { // 事件处理函数的名字
            // this 表示当前 new 出来的 vm 实例对象，
            // 通过 this 可以访问到 data 中的数据
            this.count += 1
        }
    },
})
```

- 事件绑定的简写形式
  - 由于 v-on 指令在开发中使用频率非常高，因此，vue 官方为其提供了简写形式（简写为英文的 @ ）。

```
<div id="app">
    <h3>count 的值为：{{count}}</h3>

    <!-- 完整写法 -->
    <button v-on:click="addCount">+1</button>

    <!-- 简写形式，把 v-on: 简写为 @ 符号 -->
    <!-- 如果事件处理函数中的代码足够简单，只有一行代码，则可以简写到行内 -->
    <button @click="count += 1">+1</button>
</div>
```

- 事件参数对象
  - 在原生的 DOM 事件绑定中，可以在事件处理函数的形参处，接收事件参数对象 event。同理，在 v-on 指令（简写为 @ ）所绑定的事件处理函数中，同样可以接收到事件参数对象 event，示例代码如下：

```
<h3>count 的值为：{{count}}</h3>
<button v-on:click="addCount">+1</button>
// ----------------------分割线---------------------------
methods: {
    addCount(e) { // 接收事件参数对象 event，简写为 e
        const nowBgColor = e.target.style.backgroundColor
        e.target.style.backgroundColor = nowBgColor === 'red' ? '' : 'red'
        this.count += 1
    }
}
```

- 绑定事件并传参
  - 在使用 v-on 指令绑定事件时，可以使用 ( ) 进行传参，示例代码如下：

```
<h3>count 的值为：{{count}}</h3>
<button @click="addNewCount(2)">+2</button>
// ----------------------分割线----------------
methods: {
    // 在形参处用 step 接收传递过来的参数值
    addNewCount(step) {
        this.count += step
    }
}
```

- $event
  - $event 是 vue 提供的特殊变量，用来表示原生的事件参数对象 event。$event 可以解决事件参数对象 event被覆盖的问题。示例用法如下：

```
<h3>count 的值为：{{count}}</h3>
<button @click="addNewCount(2, $event)">+2</button>
// ----------------------分割线--------------------------
methods: {
    // 在形参处用 e 接收传递过来的原生事件参数对象 $event
    addNewCount(step, e) {
        const nowBgColor = e.target.style.backgroundColor
        e.target.style.backgroundColor = nowBgColor === 'cyan' ? '' : 'cyan'
        this.count += step
    }
}
```

- 事件修饰符
  - 在事件处理函数中调用 event.preventDefault() 或 event.stopPropagation() 是非常常见的需求。因此，vue 提供了事件修饰符的概念，来辅助程序员更方便的对事件的触发进行控制。常用的 5 个事件修饰符如下：

事件修饰符	说明
.prevent	阻止默认行为（例如：阻止 a 连接的跳转、阻止表单的提交等）
.stop	阻止事件冒泡
.capture	以捕获模式触发当前的事件处理函数
.once	绑定的事件只触发1次
.self	只有在 event.target 是当前元素自身时触发事件处理函数

- 语法格式如下：

```
<!-- 触发 click 点击事件时，阻止 a 链接的默认跳转行为 -->
<a href="https://www.baidu.com" @click.prevent="onLinkClick">百度首页</a>
```

- 按键修饰符
  - 在监听键盘事件时，我们经常需要判断详细的按键。此时，可以为键盘相关的事件添加按键修饰符，例如：

```
<!-- 只有在 `key` 是 `Enter` 时调用 `vm.submit()` -->
<input @keyup.enter="submit">

  <!-- 只有在 `key` 是 `Esc` 时调用 `vm.clearInput()` -->
<input @keyup.esc="clearInput">
```

- **vue的指令与过滤器——双向绑定指令**
  - vue 提供了 v-model 双向数据绑定指令，用来辅助开发者在不操作 DOM 的前提下，快速获取表单的数据。

```
<p>用户名是：{{username}}</p>
<input type="text" v-model="username" />

<p>选中的省份是：{{province}}</p>
<select v-model="province">
    <option value="">请选择</option>
    <option value="1">北京</option>
    <option value="2">河北</option>
    <option value="3">黑龙江</option>
</select>
```

- v-model 指令的修饰符
  - 为了方便对用户输入的内容进行处理，vue 为 v-model 指令提供了 3 个修饰符，分别是：

修饰符	作用	示例
.number	自动将用户的输入值转为数值类型	<input v-model.number="age" />
.trim	自动过滤用户输入的首尾空白字符	<input v-model.trim="msg" />
.lazy	在“change”时而非“input”时更新	<input v-model.lazy="msg" />

  - 示例用法如下：

```
<input type="text" v-model.number="n1"> +
<input type="text" v-model.number="n2"> =
<span>{{n1 + n2}}</span>
```

- **vue的指令与过滤器——条件渲染指令**
  - 条件渲染指令用来辅助开发者按需控制 DOM 的显示与隐藏。条件渲染指令有如下两个，分别是：
    - v-if
    - v-show
  - 示例用法如下：

```
<div id="app">
    <p v-if="networkState === 200">请求成功 --- 被 v-if 控制</p>
    <p v-show="networkState === 200">请求成功 --- 被 v-show 控制</p>
</div>
```

  - v-if 和 v-show 的区别
    - 实现原理不同：
      - v-if 指令会动态地创建或移除 DOM 元素，从而控制元素在页面上的显示与隐藏；
      - v-show 指令会动态为元素添加或移除 style="display: none;" 样式，从而控制元素的显示与隐藏；
    - 性能消耗不同：
      - v-if 有更高的切换开销，而 v-show 有更高的初始渲染开销。因此：
      - 如果需要非常频繁地切换，则使用 v-show 较好
      - 如果在运行时条件很少改变，则使用 v-if 较好
  - v-else
    - v-if 可以单独使用，或配合 v-else 指令一起使用：

```
<div v-if="Math.random() > 0.5">
  随机数大于 0.5
</div>
<div v-else>
  随机数小于或等于 0.5
</div>
```

    - 注意：v-else 指令必须配合 v-if 指令一起使用，否则它将不会被识别！
  - v-else-if
    - v-else-if 指令，顾名思义，充当 v-if 的“else-if 块”，可以连续使用：

```
<div v-if="type === 'A'">优秀</div>
<div v-else-if="type === 'B'">良好</div>
<div v-else-if="type === 'C'">一般</div>
<div v-else>差</div>
```

- 注意：v-else-if 指令必须配合 v-if 指令一起使用，否则它将不会被识别！

- **vue的指令与过滤器——列表渲染指令**
  - vue 提供了 v-for 列表渲染指令，用来辅助开发者基于一个数组来循环渲染一个列表结构。v-for 指令需要使用 item in list 形式的特殊语法，其中：
    - list 是待循环的数组
    - item 是被循环的每一项

```
data: {
  list: [ // 列表数据
    { id: 1, name: 'zs' },
    { id: 2, name: 'ls' }
  ]
}
// ----------------分割线--------------------
<ul>
  <li v-for="item in list">姓名是：{{item.name}}</li>
</ul>
```

  - v-for 中的索引
    - v-for 指令还支持一个可选的第二个参数，即当前项的索引。语法格式为 (item, index) in items，示例代码如下：

```
data: {
  list: [ // 列表数据
    { id: 1, name: 'zs' },
    { id: 2, name: 'ls' }
  ]
}
// ----------------分割线--------------------
<ul>
  <li v-for="(item, index) in list">索引是：{{index}}，姓名是：{{item.name}}</li>
</ul>
```

    - 注意：v-for 指令中的 item 项和 index 索引都是形参，可以根据需要进行重命名。例如 (user, i) in userlist
  - 使用 key 维护列表的状态
    - 当列表的数据变化时，默认情况下，vue 会尽可能的复用已存在的 DOM 元素，从而提升渲染的性能。但这种默认的性能优化策略，会导致有状态的列表无法被正确更新。

- 为了给 vue 一个提示，以便它能跟踪每个节点的身份，从而在保证有状态的列表被正确更新的前提下，提升渲染的性能。此时，需要为每项提供一个唯一的 key 属性：

```
<!-- 用户列表区域 -->
<ul>
  <!-- 加 key 属性的好处： -->
  <!-- 1. 正确维护列表的状态 -->
  <!-- 2. 复用现有的 DOM 元素，提升渲染的性能 -->
  <li v-for="user in userlist" :key="user.id">
    <input type="checkbox" />
    姓名：{{user.name}}
  </li>
</ul>
```

- key 的注意事项
  - key 的值只能是字符串或数字类型
  - key 的值必须具有唯一性（即：key 的值不能重复）
  - 建议把数据项 id 属性的值作为 key 的值（因为 id 属性的值具有唯一性）
  - 使用 index 的值当作 key 的值没有任何意义（因为 index 的值不具有唯一性）
  - 建议使用 v-for 指令时一定要指定 key 的值（既提升性能、又防止列表状态紊乱）

- **vue的指令与过滤器——过滤器**
  - 过滤器（Filters）是 vue 为开发者提供的功能，常用于文本的格式化。过滤器可以用在两个地方：插值表达式和 v-bind 属性绑定。
  - 过滤器应该被添加在 JavaScript 表达式的尾部，由“管道符”进行调用，示例代码如下：

```
<!-- 在双花括号中通过“管道符”调用 capitalize 过滤器，对 message 的值进行格式化 -->
<p>{{ message | capitalize }}</p>

<!-- 在 v-bind 中通过“管道符”调用 formatId 过滤器，对 rawId 的值进行格式化 -->
<div v-bind:id="rawId | formatId"></div>
```

  - 定义过滤器
    - 在创建 vue 实例期间，可以在 filters 节点中定义过滤器，示例代码如下：

```
const vm = new Vue({
  el: '#app',
  data: {
    message: 'hello vue.js',
    info: 'title info'
  },
  filters: {          // 在 filters 节点下定义“过滤器”
    capitalize(str) { // 把首字母转为大写的过滤器
      return str.charAt(0).toUpperCase() + str.slice(1)
    }
  }
})
```

- 私有过滤器和全局过滤器
  - 在 filters 节点下定义的过滤器，称为“私有过滤器”，因为它只能在当前 vm 实例所控制的 el 区域内使用。如果希望在多个 vue 实例之间共享过滤器，则可以按照如下的格式定义全局过滤器：

```
// 全局过滤器 - 独立于每个 vm 实例之外
// Vue.filter() 方法接收两个参数：
//   第 1 个参数，是全局过滤器的“名字”
//   第 2 个参数，是全局过滤器的“处理函数”
Vue.filter('capitalize', (str) => {
  return str.charAt(0).toUpperCase() + str.slice(1) + '~~'
})
```

- 连续调用多个过滤器
  - 过滤器可以串联地进行调用，例如：

```
<!-- 把 message 的值，交给 filterA 进行处理 -->
<!-- 把 filterA 处理的结果，再交给 filterB 进行处理 -->
<!-- 最终把 filterB 处理的结果，作为最终的值渲染到页面上 -->
{{ message | filterA | filterB }}
```

  - 示例代码如下：

```
<!-- 串联调用多个过滤器 -->
<p>{{text | capitalize | maxLength}}</p>

// 全局过滤器 - 首字母大写
Vue.filter('capitalize', (str) => {
  return str.charAt(0).toUpperCase() + str.slice(1) + '~~'
})

// 全局过滤器 - 控制文本的最大长度
Vue.filter('maxLength', (str) => {
  if (str.length <= 10) return str
  return str.slice(0, 11) + '...'
})
```

- 过滤器传参
  - 过滤器的本质是 JavaScript 函数，因此可以接收参数，格式如下：

```
<!-- arg1 和 arg2 是传递给 filterA 的参数 -->
<p>{{ message | filterA(arg1, arg2) }}</p>

// 过滤器处理函数的形参列表中：
//   第一个参数：永远都是"管道符"前面待处理的值
//   从第二个参数开始，才是调用过滤器时传递过来的 arg1 和 arg2 参数
Vue.filter('filterA', (msg, arg1, arg2) => {
  // 过滤器的代码逻辑...
})
```

  - 示例代码如下：

```
<!-- 调用 maxLength 过滤器时传参 -->
<p>{{text | capitalize | maxLength(5)}}</p>

// 全局过滤器 - 首字母大写
Vue.filter('capitalize', (str) => {
  return str.charAt(0).toUpperCase() + str.slice(1) + '~~'
})

// 全局过滤器 - 控制文本的最大长度
Vue.filter('maxLength', (str, len = 10) => {
  if (str.length <= len) return str
  return str.slice(0, len) + '...'
})
```

- 过滤器的兼容性
  - 过滤器仅在 vue 2.x 和 1.x 中受支持，在 vue 3.x 的版本中剔除了过滤器相关的功能。
  - 在企业级项目开发中：
    - 如果使用的是 2.x 版本的 vue，则依然可以使用过滤器相关的功能
    - 如果项目已经升级到了 3.x 版本的 vue，官方建议使用计算属性或方法代替被剔除的过滤器功能
  - 具体的迁移指南，请参考 vue 3.x 的官方文档给出的说明：
  - https://v3.vuejs.org/guide/migration/filters.html#migration-strategy